

Here's how we structured it:

- React Router handles client-side routing.
- Axios manages all API calls, with an interceptor to attach the user's token.
- React Context is used to manage shared app state, such as logged-in user info and role-specific view rendering.

We also set up centralised error handling, global toasts for feedback, and route guards to redirect unauthorised users, especially useful when switching between roles.

Real-time support with

WebSockets: Though we've already touched on this in our frontend section, it's worth noting here: our architecture includes a WebSocket channel using Socket.IO for real-time updates. This

layer is particularly important for features like:

- Appointment status changes
- Live chat within support groups
- Notifications when new documents are published

The WebSocket connection is established during login and lives throughout the session, allowing us to push updates without the frontend needing to poll the server continuously.

Figure 4 outlines how the frontend, API gateway, modular services, and database collections interact across both HTTP (REST) and WebSocket layers. This modular setup made debugging and feature updates much easier. For example, when we wanted to tweak the group chat logic, we didn't have to touch the appointment code at all. Over time, we realised this architecture

not only improved performance and security, but also helped onboard new contributors quickly since each service is like its own mini project.

Technology stack

SereneCare is built entirely using open source technologies that offer flexibility, performance, and scalability. The platform adopts a modern full-stack approach that ensures seamless communication between frontend and backend components while maintaining a strong focus on user experience and security.

One of the early decisions we made with SereneCare was to stick with tools we were already comfortable with — open source, battle-tested, and flexible enough to grow with the platform. Our aim wasn't to experiment with flashy frameworks but to build something

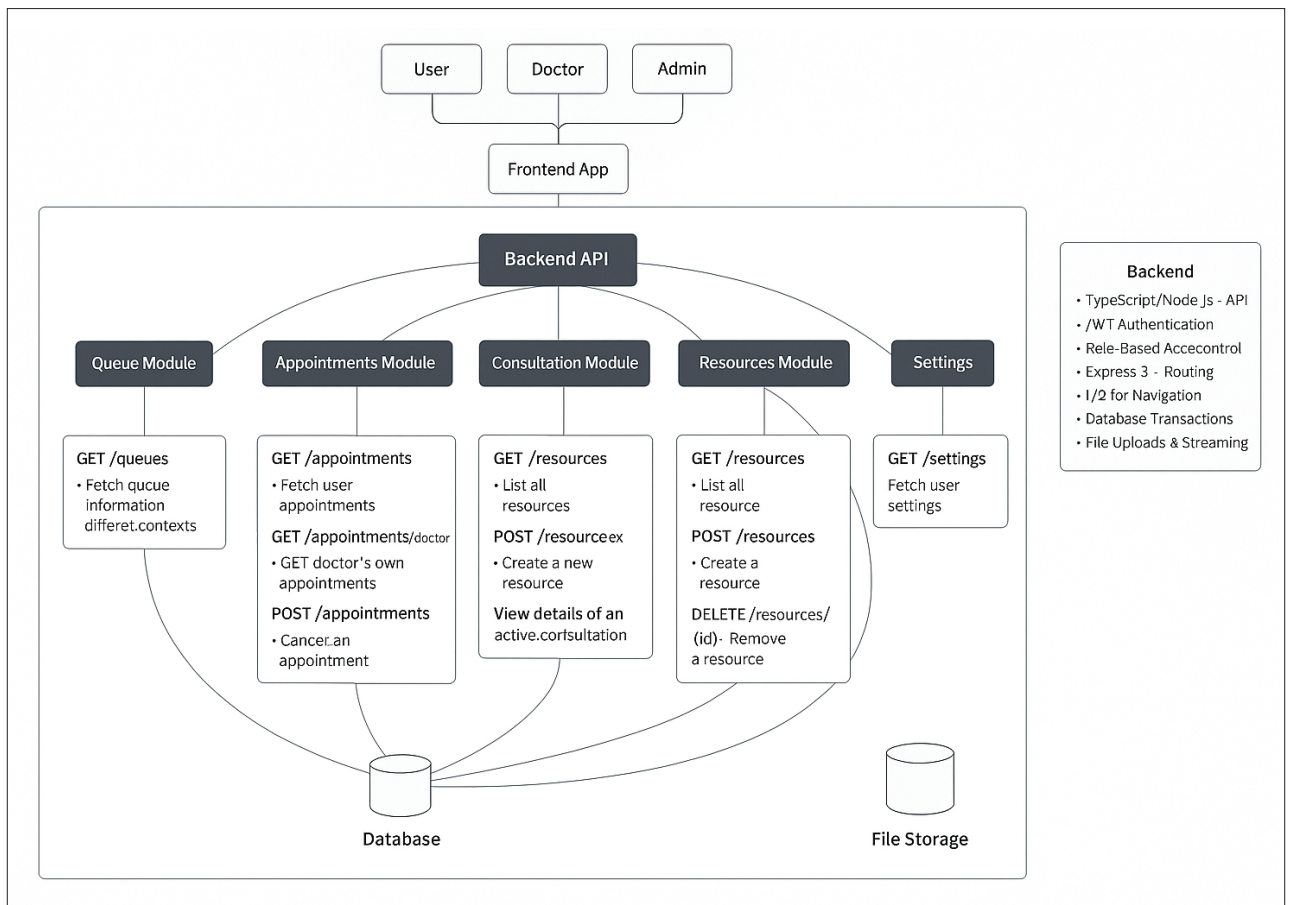


Figure 4: High-level system architecture of SereneCare